



Introduzione a typescript

Programmazione

TypeScript è un linguaggio di programmazione open source sviluppato come estensione tipizzata di JavaScript.

Mantiene la sintassi e il modello di esecuzione di JavaScript, ma introduce un sistema di tipi statici che permette di dichiarare esplicitamente la struttura delle variabili, dei parametri, dei valori restituiti dalle funzioni e degli oggetti.

Un file TypeScript utilizza generalmente l'estensione .ts e non viene eseguito direttamente dal browser, ma deve essere trasformato in JavaScript attraverso un processo chiamato compilazione o transpilazione.



Il sistema di tipizzazione di TypeScript consente di individuare molti errori durante la fase di sviluppo, prima dell'esecuzione del programma.

Il compilatore verifica, ad esempio, che una funzione riceva parametri del tipo previsto, che una proprietà esista all'interno di un oggetto e che il valore restituito sia compatibile con la dichiarazione effettuata.

La tipizzazione può essere esplicita oppure inferita automaticamente dal compilatore in base al valore assegnato.



TypeScript supporta i principali paradigmi di programmazione disponibili in JavaScript, inclusi programmazione funzionale, asincrona e orientata agli oggetti.

Nel paradigma OOP mette a disposizione classi, interfacce, ereditarietà, classi astratte, modificatori di accesso, generics e implementazione dei contratti.

Questi strumenti facilitano la progettazione di applicazioni modulari, scalabili e caratterizzate da responsabilità chiaramente definite.



Le interfacce e i type alias permettono di descrivere la struttura dei dati utilizzati dall'applicazione.

Un'interfaccia può definire le proprietà e i metodi che un oggetto deve possedere, mentre i generics consentono di creare componenti riutilizzabili mantenendo il controllo sul tipo dei dati elaborati.

TypeScript utilizza inoltre un sistema di tipizzazione strutturale, nel quale la compatibilità tra tipi dipende principalmente dalla loro struttura e non esclusivamente dal nome dichiarato.



Il compilatore TypeScript, configurato normalmente attraverso il file tsconfig.json, permette di specificare la versione JavaScript di destinazione, il sistema di moduli, le regole di controllo dei tipi, le cartelle di input e output e il livello di severità della compilazione.

L'opzione strict, ad esempio, abilita controlli più rigorosi e riduce la possibilità che valori non definiti, nulli o incompatibili raggiungano la fase di esecuzione.



TypeScript viene utilizzato principalmente nello sviluppo di applicazioni web frontend, backend Node.js, API, applicazioni desktop e progetti multiplatforma.

Framework come Angular lo adottano come linguaggio principale, mentre React, Vue, Express e NestJS possono integrarlo per migliorare la manutenibilità del codice, il supporto degli strumenti di sviluppo, il refactoring e la collaborazione all'interno di team che lavorano su codebase di grandi dimensioni.



- **Tipizzazione statica**

TypeScript permette di dichiarare e controllare i tipi di variabili, parametri, proprietà e valori restituiti, individuando molti errori prima dell'esecuzione del programma.

- **Compatibilità con JavaScript**

ogni codice JavaScript valido può essere utilizzato in TypeScript, consentendo di introdurre il linguaggio gradualmente anche all'interno di progetti già esistenti.



- **Supporto alla programmazione orientata agli oggetti:**

TypeScript include classi, interfacce, ereditarietà, classi astratte, modificatori di accesso e generics per progettare software modulare e scalabile.

- **Compilazione configurabile**

il codice TypeScript viene trasformato in JavaScript attraverso il compilatore tsc, configurabile tramite tsconfig.json per definire versione di destinazione, moduli, controlli e livello di severità.



```
1. let nome: string = "Marco";  
2. ATTENZIONE ---> let eta: number = 30;  
3. let isDocente: boolean = true;  
4.  
5. console.log(nome);  
6. console.log(eta);  
7. console.log(isDocente);
```

In questo esempio vengono dichiarate tre variabili con tipi differenti; la variabile nome può contenere esclusivamente valori testuali, eta può contenere valori numerici e isDocente può assumere solamente i valori booleani true oppure false.



TypeScript controlla la compatibilità tra il valore assegnato e il tipo dichiarato durante la compilazione.

```
1.function somma(numero1: number, numero2: number): number {  
2.    return numero1 + numero2; }  
3.  
4.const risultato: number = somma(10, 5);  
5.  
6.console.log(risultato);
```

La funzione somma riceve due parametri di tipo number e restituisce un valore dello stesso tipo.

La sintassi : number posizionata dopo le parentesi indica il tipo del valore restituito dalla funzione. La variabile risultato conterrà quindi il valore numerico 15.



Questa chiamata potrebbe produrre un errore di compilazione perché il secondo parametro è una stringa, mentre la funzione richiede due valori numerici.

Le principali caratteristiche semantiche di TypeScript riguardano il modo in cui il linguaggio attribuisce significato ai dati, alle funzioni e alle strutture del programma.

TypeScript utilizza una tipizzazione statica, controllando prima dell'esecuzione che ogni valore sia compatibile con il tipo dichiarato.

Supporta inoltre l'inferenza dei tipi, che permette al compilatore di dedurre automaticamente il tipo di una variabile, la tipizzazione strutturale, basata sulla forma degli oggetti anziché sul loro nome, e i tipi unione, che consentono a una variabile di accettare più tipi controllati.



```
1. let messaggio = "Benvenuto"; // Tipo string dedotto  
2.  
3. let identificatore: string | number;  
4.  
5. identificatore = 25;  
6. identificatore = "USR-25";
```



Un'altra caratteristica importante è la possibilità di descrivere semanticamente gli oggetti tramite interface e type.

Questi strumenti stabiliscono quali proprietà e metodi devono essere presenti, migliorando la leggibilità e la coerenza del codice.

TypeScript supporta anche proprietà opzionali, generics, literal type, enumerazioni e controlli di tipo, permettendo di rappresentare in modo preciso le regole del dominio applicativo.



```
1.interface Utente {  
2.  nome: string;  
3.  eta: number;  
4.  email?: string;  
5.}  
6.  
7.function descriviUtente(utente: Utente): string {  
8.  return `${utente.nome} ha ${utente.eta} anni`;   
9.}  
10.  
11.const docente: Utente = {  
12.  nome: "Marco",  
13.  eta: 35  
14.};  
15.  
16.console.log(descriviUtente(docente));
```



Buon DAVANTE a tutti

